MIDTERM PROJECT REPORT

Gray Code
&
Customized Fitness Function
Implementations in MATLAB

TABLE OF CONTENTS

Semanur Bilada
Computer Engineering

1. Introduction

This report is submitted as a midterm project for the Genetic Algorithms course. It covers two fundamental topics addressed in the scope of the course: the design and implementation of fitness function, and the representation of solutions using Gray Code encoding. Both implementations are carried out in MATLAB. The report includes code explanations, conversion examples and program outputs for each topic.

2. Gray Code

Gray Code is a binary numeral system where two successive values differ in only one bit. Unlike standard binary encoding where multiple bits can change simultaneously between consecutive numbers, Gray Code ensures a single-bit transition at each step. This property makes it particularly useful in genetic algorithms where it reduces the risk of large jumps in the solution space during mutation and crossover operations.

Conversion Rule (Binary to Gray Code): As shown in Figure 1, each Gray Code bit is derived by applying the XOR operation between the current binary bit and the previous one. The Most Significant Bit (MSB) is copied directly.

```
Binary:    1  0  1  1
           |  |  |  |
G1 = B1          ->  1 (MSB)
G2 = B1 XOR B2   ->  1 XOR 0 = 1
G3 = B2 XOR B3   ->  0 XOR 1 = 1
G4 = B3 XOR B4   ->  1 XOR 1 = 0

Gray Code: 1  1  1  0
```

Figure 1: Bit-by-bit derivation of Gray Code from binary representation 1011, illustrating the XOR-based conversion rule applied at each bit position

The following script (Figure 2) generates a 4-bit Gray Code table for all decimal values from 0 to 15. The conversion is performed in a single line using MATLAB's built-in *bitxor* and *bitshift* functions.

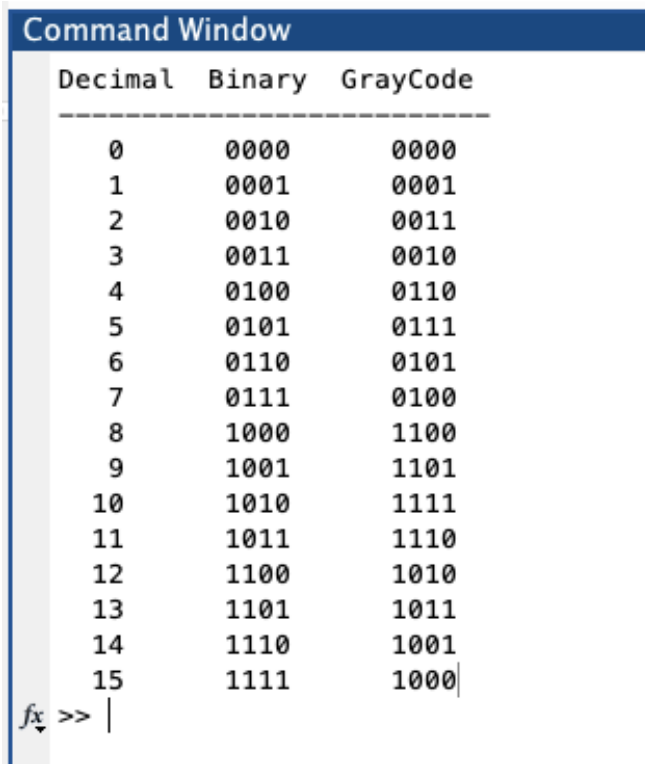
```
%% Gray Code
clc; clear;

n_bits = 4;

fprintf('Decimal  Binary  GrayCode\n');
fprintf('-------------------------\n');

for d = 0 : 2^n_bits - 1
    gray = bitxor(d, bitshift(d, -1));  % one line gray code
    fprintf('  %2d      %s      %s\n', d, dec2bin(d, n_bits), dec2bin(gray, n_bits));
end
```

Figure 2: MATLAB implementation of Gray Code



The core formula $bitxor(d, bitshift(d, -1))$ works as follows: shifting d one bit to the right aligns each bit with its predecessor and applying XOR produces the Gray Code equivalent in a single operation — a direct implementation of the G(i) = B(i) XOR B(i−1) rule.

Figure 3: Output of Gray Code sample in MATLAB

3.Customized Fitness Function

In this study, a customized fitness function is implemented to evaluate solutions generated by a genetic algorithm for a simple network topology optimization problem. The main goal of the fitness function is to guide the genetic algorithm toward node placements that result in short communication distances while maintaining basic connectivity between nodes.

Each individual solution handled by the genetic algorithm is represented as a vector containing the two-dimensional coordinates of network nodes. For five nodes, the solution vector includes ten variables corresponding to the *x* and *y* coordinates of each node.

3.1.Fitness Function Structure

The fitness function is implemented in MATLAB as the **fitness_network** function. The first step in the function reshapes the one-dimensional input vector into a two-dimensional coordinate matrix where each row represents the position of a network node (Figure 4).

After reshaping the data, the function calculates the distances between all unique pairs of nodes using nested loops. For each node pair, the Euclidean distance is computed and added to a variable representing the total distance in the network. This total distance reflects the overall communication cost and is intended to be minimized by the genetic algorithm.

In addition to distance minimization, a simple connectivity rule is applied. If the distance between two nodes exceeds 40 units, a fixed penalty value is added to the fitness score. This penalty discourages solutions where nodes are placed too far apart and represents a soft connectivity constraint within the fitness function.

The final fitness value is calculated as the sum of the total distance and the accumulated penalty values. Since the genetic algorithm minimizes the fitness value, lower scores indicate better network configurations.

fitness_custom.m

```
%% Run
fitnessFcn = @fitness_network;
[x_best, fval] = ga(fitnessFcn, nvars, [], [], [], [], lb, ub, [], options);

%% Fitness Function
% Network Topology Optimization Fitness Function
% Input: x = [x1, y1, x2, y2, x3, y3, ...] (coordinates)
% Output: score = fitness value (target to reduce)
% Target:
% 1. Minimize total distance (energy saving)
% 2. Nodes should not be more than 40 units apart (connectivity)
function score = fitness_network(x)
    % Convert coordinates to 2D shape: [x1 y1; x2 y2; ...]
    coords = reshape(x, [], 2);
    n = size(coords, 1);     % Node count

    totalDistance = 0;       % Total distance (minimize)
    connectivityPenalty = 0;

    % Calculate the distance between all node pairs
    for i = 1:n
        for j = i+1:n
            % Euclidean distance: sqrt((x2-x1)² + (y2-y1)²)
            distance = norm(coords(i,:) - coords(j,:));

            % Add to total distance (to minimize)
            totalDistance = totalDistance + distance;

            % Connectivity constraint: Penalty if it's too far
            if distance > 40
                % A penalty of 100 for each unit exceeding 40 units
                connectivityPenalty = connectivityPenalty + 100;
            end
        end
    end

    % Total fitness score
    score = totalDistance + connectivityPenalty;
    % Low score = good network
    % High score = bad network (too far away or penalized)
end
```

Figure 4: MATLAB implementation of Customized Fitness Function

3.2. Output of Fitness Function Structure and Interpretation

The genetic algorithm is executed using MATLAB's built-in ga function. The command window output (Figure 5) shows that the best and mean fitness values decrease regularly over generations. This behavior indicates that the genetic algorithm successfully improves the population by selecting and evolving better solutions over time.

As the algorithm progresses, solutions with large distances and connectivity penalties are gradually eliminated. In the final generations, the fitness value approaches zero indicating that the algorithm has found a solution that minimizes the defined objectives.

The final solution places all nodes at the same coordinates. This result produces zero total distance and no connectivity penalties which corresponds to the global minimum of the defined fitness function. Since no constraint is included to prevent nodes from overlapping, this solution is mathematically optimal under the current fitness formulation.

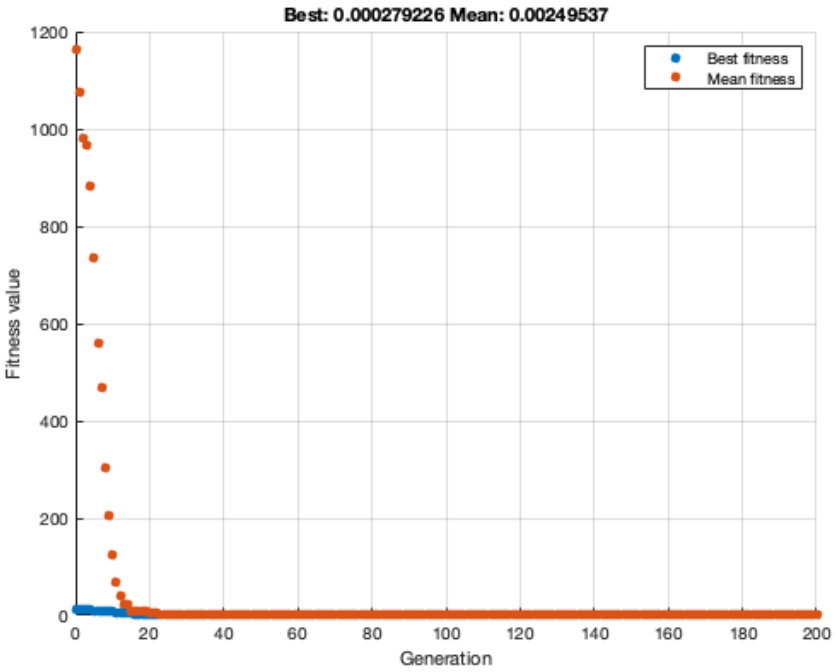


Figure 5: Output of Customized Fitness Function in MATLAB

This result demonstrates that the customized fitness function works as intended and that the genetic algorithm correctly minimizes the defined objectives. The outcome also highlights the importance of fitness function design as the absence of a minimum separation constraint allows node overlap in the optimal solution. This initial model provides a clear and simple baseline that can be extended with additional constraints in future studies such as final stage of the project.

4. Conclusion

In this midterm project, two core concepts of genetic algorithms were studied and implemented using MATLAB. The first part focused on solution representation through Gray Code encoding where the conversion process was explained and demonstrated with sample inputs and outputs.

The second part of the project examined the design of a customized fitness function. A problem-specific fitness function was developed to evaluate network node placements based on total distance and basic connectivity constraints. The results obtained from the genetic algorithm show that the defined fitness function is successfully minimized and the algorithm behaves as expected.

This project represents the completion of the midterm stage of the overall study. In the final stage, the fitness function and network model will be further improved by introducing additional constraints such as preventing node overlap and incorporating more realistic network requirements. Overall, this project helped reinforce the theoretical concepts of genetic algorithms through practical MATLAB implementations.